SKYROUTE DBMS MINI PROJECT

SkyRoute: Airline Booking, Baggage & Staff Operations System

DBMS Mini Project -- Database Management System (CS/AI/AIM/DS202)

Database: skyroute_db | Tables: 16 | MySQL 8.0 (InnoDB, utf8mb4)

Seed volume: 336 dated departures, ~34,000 bookings, ~42,000 baggage records (deterministic generator, seed 42).

Abstract

Airline stations that manage bookings, seat allocation, gate assignment and baggage handling through manual registers and disconnected station files face five recurring failures: duplicate seat bookings when two agents sell concurrently, gate assignment clashes between flights, miscalculated baggage fees above the free allowance, inconsistent data across stations with no single source of truth, and overselling beyond aircraft capacity with no automated guard. SkyRoute is a centralized relational database for five Indian airports (HYD, BLR, BOM, DEL, MAA) and three airlines that automates scheduling, reservations, baggage tracking, payments and check-in. The schema separates timetable templates (Flights) from dated departures (Departures) so one scheduled flight owns an unlimited number of dated departures; models Terminal and Gate infrastructure per airport; and organises staff with a organises staff in role tables sharing each employee's key (Employees 1-1 Pilot, Crew_Member and Ground_Staff). Integrity rules live inside the database rather than in any single application: a unique seat-per-departure key blocks double booking, a before-insert trigger blocks overselling past aircraft capacity, composite gate keys block gate clashes, a trigger blocks impossible routes (HYD to HYD), and check constraints block non-positive baggage weights and

negative prices. A stored
procedure books a seat and opens its payment atomically; two reporting views
serve the departure board and the booking receipt including the computed
overweight-baggage fee (20 kg free per bag, Rs 500 per excess kg); transaction
demonstrations prove atomicity and isolation under the REPEATABLE READ level;
and four workload-driven indexes, each verified with a before/after EXPLAIN
pair, keep analytical queries fast at a demonstrated volume of 336 departures
and roughly thirty-four thousand bookings. Every problem-statement item is closed by
a named database mechanism, every mechanism carries a test in the validation
protocol, and the design is carried into a Flask staff-portal web application
backed by MySQL through mysql-connector-python -- traveler login,
per-departure manifests, full create/read/update/delete coverage -- where
every insert, update and delete lands visibly in the database server, reusing
the same integrity rules in application code.

Step 1: Title Selection

SkyRoute: Airline Booking, Baggage & Staff Operations System. The title was chosen to
reflect the real-world problem solved -- reservations, baggage handling and
staff operations for scheduled airline operations across multiple airports --
rather than a single-route shuttle, a generic "airport database", or any
"Airport Management System" style title. The subtitle carried
through all deliverables is "one database for every station", which states the
centralisation objective in a single line.

Step 2: Problem Statement

The study began from the manual and single-station booking workflows observed
at Hyderabad (HYD) and Bengaluru (BLR): paper seat charts per flight, gate
allocations written on whiteboards, baggage fees computed by hand at the
counter, and each station keeping its own passenger lists. Five concrete
failures follow from that way of working, and each maps to exactly one
database mechanism built in Steps 7 through 9, so that every problem item is
provably closed rather than merely addressed.

- Duplicate seat booking on the same departure when two agents sell
concurrently. Paper charts cannot serialize two sellers; the database can.
Closed by UNIQUE(departure_id, seat_number) on Bookings, enforced by the
storage engine itself, plus a capacity trigger as the second line of defence.
- Gate assignment clashes written down wrongly between flights. Two departures

end up on the same gate at overlapping times because nothing prevents the double entry. Closed by UNIQUE(terminal_id, gate_number) on Gates together with gate rotation in scheduling, so the same physical gate is one row.

- Baggage fees miscalculated when weight exceeds the 20 kg free allowance. Counter arithmetic varies by agent. Closed by encoding the fee rule exactly once, in booking_receipt_view: 20 kg free per bag, Rs 500 per excess kg, computed identically on every receipt.
- No single database for flights, passengers and bookings across stations. HYD and BLR lists disagree. Closed by one skyroute_db serving all five airports, with airports as the root master table.
- No automated guard when a departure sells more seats than the aircraft holds. Closed by trg_booking_capacity, a before-insert trigger that compares seats sold against the assigned aircraft's capacity and refuses the excess row with SQLSTATE 45000.

Step 3: Objectives

- Automate flight scheduling and gate allocation across five airports.
- Make double seat booking and overselling impossible at the storage-engine level, independent of which application writes the rows.
- Compute baggage fees deterministically from weighed kilograms, identically on every receipt.
- Retrieve departure boards, passenger manifests, revenue, load factors and gate utilisation with single queries.
- Guarantee integrity with least-privilege access control (admin, staff and viewer roles) verified against the live grant tables.
- Prove performance at volume with workload-driven indexing and before/after query-cost analysis.
- Carry the design into a working web application with full CRUD coverage as the ES202 deliverable.

Step 4: Requirements Gathering

Functional requirements: add, edit and view airports, airlines, aircraft, timetable flights and dated departures; assign aircraft and gates to each dated departure; reserve seats and create bookings; log baggage weight per booking; record payments and check-ins; manage staff records and roles; suspend or reactivate timetable flights.

Non-functional requirements: query responses must stay interactive at tens of

thousands of rows; concurrent sellers must never corrupt seat allocation (isolation plus engine-level uniqueness); access must follow least privilege; every business rule must be demonstrable live in under three minutes.

Out of scope, by explicit decision: crew rostering links (staff exist as records, daily assignment is an application concern), runways as a separate entity (gate plus terminal carry the clash-prevention story), and fare pricing engines (price is stored per booking; revenue is aggregated, not computed).

Step 5: Entity-Relationship Modeling

Entities: Airport, Airline, Aircraft, Flight, Departure, Terminal, Gate, Passenger, Booking, Baggage (weak entity, double-bordered), Payment, Check_In, Employees with three role tables (Pilots, Crew_Members, Ground_Staff), each 1-1 on the shared employee key.

Relationships and cardinalities: Airline 1-N Aircraft (a fleet belongs to one airline); Flight N-1 Airports in two roles, departure and arrival; Flight 1-N Departures (one timetable row owns unlimited dated departures); Airport 1-N Terminals 1-N Gates; Passenger 1-N Bookings N-1 Departures (a booking is the associative resolution of one passenger and one departure, carrying seat, class and price); Booking 1-N Baggage through an identifying relationship (bag 2 of booking 817 has no meaning without the booking); Booking 1-1 Payment and 1-1 Check_In; Employees 1-1 each role table through the shared key (a licence row cannot exist without its employee row). Keys, cardinality and participation were fixed on the diagram before any CREATE TABLE was written, using Chen notation: rectangles for entities, diamonds for relationships, ovals for attributes, double borders for the weak entity and its identifying relationship. The final diagram is delivered as SkyRoute_ER.drawio with a PNG/PDF export path.

Step 6: Schema Design (Relational Model)

Table	Primary key	Rule that matters
Airports	airport_id	UNIQUE iata, name
Airlines	airline_id	UNIQUE name, iata
Aircraft	aircraft_id	UNIQUE tail; CHECK cap 1-850
Flights	flight_id	UNIQUE number; dep <> arr airports
Departures	departure_id	UNIQUE(flight, dep datetime)
Terminals / Gates	own ids	UNIQUE per parent (anti-clash)
Passengers	passenger_id	UNIQUE passport, email
Bookings	booking_id	UNIQUE(departure, seat)

Baggage	(booking, seq)	weak entity; 0 < kg <= 100
Payments / Check-Ins	own ids	UNIQUE booking (one-to-one)
Employees + role tables	employee_id	role PK = employee PK

The ER model converts to 16 InnoDB tables. Surrogate integer primary keys keep foreign keys narrow at scale; natural business keys (IATA codes, tail numbers, licences, emails, transaction references) carry UNIQUE constraints instead of serving as keys. Master tables (Airports, Airlines, Aircraft) are referenced, never copied: an airline name lives once, not inside every flight row.

Normalisation account: first normal form holds because repeating groups were removed into masters and every attribute is atomic (no comma-separated seat lists anywhere); second normal form holds because every non-key attribute depends on the whole key (gate status depends on gate_id alone, never partly on a booking); third normal form holds because no transitive dependencies remain (city depends on airport, and flights reference the airport instead of copying its city; airline country lives in Airlines, not in Aircraft). The design is BCNF-clean: every determinant in every table is a key, verified table by table during design review. The headline functional dependencies: airport_id determines IATA, name and city; flight_number determines airline and both airports; (flight_id, scheduled_departure) determines one departure; (departure_id, seat_number) determines one booking; (booking_id, baggage_seq) determines one bag weight; tail_number, licence, email, passport and txn_reference each determine their row; employee_id determines the person, with role facts in the role tables keyed by the same id.

Referential discipline: CASCADE where children cannot meaningfully outlive parents (bookings, bags, payments of a deleted departure vanish together); RESTRICT on airports (deleting an airport must never silently wipe timetable flights -- the engine refuses until an administrator moves them); SET NULL on optional operational assignments (removing an aircraft unassigns it from departures rather than deleting history). Domain discipline: closed ENUMs for statuses, classes, methods and roles so junk values cannot enter; CHECKs for capacity ranges, non-negative prices, positive weights capped at 100 kg, arrival after departure in both timetable and dated rows, and departure airports differing from arrival airports.

Step 7: SQL Implementation

DDL (schema/01-travel-core.sql, schema/02-staff-roles.sql): twelve travel tables plus the four-table staff hierarchy, with primary keys, foreign keys,

unique keys, checks and enums exactly as designed above, followed by the DCL: three users -- sky_admin with full privileges, sky_staff with read-write, and sky_viewer with read-only -- each verified afterwards with SHOW GRANTS against the exact database name, because a grant issued on a misspelled database name succeeds silently and protects nothing.

DML (data/generate.py, seed 42, writing data/seed_data.sql): five airports, three airlines, sixteen scheduled flights across sixteen routes, twenty-one days of dated departures (336 departures), four thousand passengers, bookings at 35 to 80 percent load factors with seats sampled without replacement (roughly thirty-four thousand bookings), matching payments at 92 percent paid, baggage at roughly 1.2 bags per booking with an 80/20 split under and over the 20 kg allowance (roughly forty-three thousand bags, giving the fee demo a real overweight tail), check-ins on about seventy percent of past departures, and eight staff with role-table rows. The generated file is machine-verified before loading: zero duplicate (departure, seat) pairs and every foreign key inside its parent range. Scale is a knob, not a rewrite: NUM_DAYS, ROUTES and the load-factor band raise volume with no design change.

DCL verification: the viewer role is shown denied on INSERT, proving least privilege rather than claiming it.

Step 8: Advanced SQL Features

Views: flight_board_view joins timetable, both airports, airline, aircraft, gate and terminal into the single departure-board query the application uses; booking_receipt_view returns price plus the computed baggage fee per booking through a scalar subquery, which keeps the view valid under ONLY_FULL_GROUP_BY where a naive GROUP BY would be rejected.

Stored procedure: MakeBooking(passenger, departure, seat, class, price, OUT id) inserts the booking and opens its Pending payment inside one transaction with an EXIT HANDLER that rolls back and re-signals on any error, so a taken seat fails the whole call atomically and never leaves half-written rows.

Trigger: trg_booking_capacity fires BEFORE INSERT ON Bookings for every insert path -- manual SQL, the procedure, the future web app -- compares seats sold against the assigned aircraft's capacity and SIGNALs SQLSTATE 45000 when the departure is full, because the rule lives in the database and not in any single client. Joins and subqueries: queries/01-joins.sql covers ten join patterns across every module (inner joins, multi-table chains, LEFT joins for optional bags and check-ins, the double airport role join, shared-key role-table

joins); queries/02-analytics.sql adds GROUP BY with HAVING, CASE aggregates, and a correlated subquery (passengers above average spend). The load-factor, revenue, gate-utilisation and fee-revenue queries are the ones that only become meaningful at volume -- trivial on three rows, decisive on thirty thousand.

Step 9: Transaction Management

All tables use InnoDB, so every demonstration runs under ACID with the default REPEATABLE READ isolation. Demo A commits a booking plus its payment together: both rows appear or neither does. Demo B takes the failure path -- a payment insert that violates a constraint -- and the rollback leaves the seat provably free afterwards, verified by re-selecting it. Demo C uses two simultaneous mysql sessions to make isolation visible: session one opens a transaction and updates a booking status without committing; session two still reads the old status; session one commits; session two re-reads and sees the new status. That invisible-then-visible sequence is isolation demonstrated live rather than defined. Script: advanced/04-transactions.sql, run block by block with screenshots.

Step 10: Indexing and Optimization

InnoDB automatically indexes single-column foreign keys at creation, so those were deliberately not re-indexed -- re-indexing them would be ceremony without effect, and the viva defence says exactly that. Four workload-driven indexes were added instead, one per observed access pattern: booking_status (filtered by every operations query over refunds and pending payments), scheduled_departure (every departure-board search is a date-range scan), (last_name, first_name) (desk lookups by family name), and weight_kg (the fee report is a range predicate over twenty kilograms). Method, per the test log: run each EXPLAIN before the CREATE INDEX, screenshot the type and rows columns, create the index, re-run EXPLAIN, screenshot again. The before/after pairs are the Step 10 deliverable. Script: advanced/05-indexes.sql.

Step 11: Testing and Validation

docs/TESTING.md records twenty-four checks in eight groups. Joins (J1-J10): all ten run clean with non-empty result sets; the LEFT JOIN manifest keeps bag-less bookings visible; the overweight list is non-empty by construction. Analytics (A1-A8): busiest routes match the scheduled frequencies; every load factor lands inside the generator's 35-80 band; fee revenue is positive. Constraints

(C-a to C-e): duplicate seat rejected with ERROR 1062; HYD-to-HYD flight rejected by trigger with ERROR 1644; zero-weight bag rejected; airport deletion refused by RESTRICT; booking deletion cascades to bags, payment and check-in with zero orphans. Procedure and trigger (T-a to T-c): MakeBooking returns an id with a matching Pending payment; MakeBooking on a taken seat inserts nothing; booking past capacity raises ERROR 45000. Transactions (D-A to D-C): commit shows both rows; rollback frees the seat; two-session read shows isolation. Indexes (I-a, I-b): EXPLAIN pairs captured; SHOW INDEX confirms exactly the four new indexes. Access control (R-a to R-c): grants match the three roles; viewer INSERT denied; grant names verified character-exact. Scale (S-a, S-b): the row-count query totals 336 departures with bookings and bags in the tens of thousands, and the daily snapshot covers all twenty-one days without gaps. Every check states its expected result before the run, so the log is both the protocol and the evidence.

Step 12: Documentation

This report (REPORT_TEXT.md as the editable source; SkyRoute_DBMS_Report.docx and SkyRoute_Report.pdf generated from it through one shared parser so the three formats cannot drift apart); README.md with run order, users, rules, layout and the screenshot workflow; TESTING.md as the validation protocol; the decision journal C1 through C13 embedded as comments in the SQL sources, tracing each design choice to its requirement; the Chen-style ER diagram with PNG/PDF export; the eleven-slide deck; and the Flask track's brief report, all versioned in the private repository with phase-by-phase commits.

Step 13: Presentation and Reporting

Deliverables: this report (within the fifteen-page limit), the eleven-slide deck (title, problem-to-mechanism map, ER, schema table, scale, advanced SQL, transactions, indexing, access control plus testing, Flask site, result), all source SQL with the deterministic generator, README, test log, and a two to three minute demo video walking the departure board, a live booking, the duplicate-seat refusal and the fee receipt. Suggested viva preparation is the per-track notes file: ten DBMS questions and seven Flask questions, each answerable in three to four sentences, covering concurrency, atomicity, indexing honesty, the table-count challenge, normalisation, isolation, scale and the fee rule.

Conclusion

Every problem-statement item is closed by a named database mechanism, every mechanism has a test with a stated expectation, and the system holds production-shaped volume while answering board, manifest, revenue, load, utilisation and fee questions in single queries. The Flask web project reuses the identical design on MySQL through mysql-connector-python: a staff portal (PIN-gated manifests, check-ins, edits) plus traveler login (unique ID, own bookings only), so every insert, update and delete lands visibly in the database server while keeping the same integrity rules visible in application code. The build order itself is the
method: masters before operations, constraints before data, rules before applications, measurement before claims.

References

- Ramakrishna R., Gehrke J. Database Management Systems, 3rd Ed, McGraw-Hill.
- Silberschatz, Korth, Sudarshan. Database System Concepts, 7th Ed.
- Elmasri R., Navathe S. Fundamentals of Database Systems, 5th Ed.
- MySQL 8.0 Reference Manual: constraints, triggers, stored programs, EXPLAIN.
- Refero Styles, Wise design system (styles.refero.design) for the web
interface visual language: forest/lime tokens, pill actions, flat surfaces.